

ADAPTIVE RAG ENGINE

Production Engineering Design Document

v1.0 — Principal Engineer Review

Stack: FastAPI · Weaviate · OpenSearch · PostgreSQL · Redis · FastHTML · LangGraph

Deployment: Hetzner VPS · Docker Compose · Nginx · GitHub Actions

Observability: OpenTelemetry · Jaeger · Prometheus · Grafana

Section 1 — System Goals

1.1 Business Goals

The Adaptive RAG Engine is designed as a flagship AI engineering portfolio project demonstrating production-grade document intelligence. The business goals are:

- Demonstrate end-to-end AI engineering capability — from ingestion through retrieval, generation, evaluation, and self-improvement
- Build a reusable platform that can be productised as a SaaS document Q&A tool
- Showcase MLOps maturity: eval pipelines, feedback loops, observability, CI/CD
- Serve as a hackathon-ready, interview-ready, and startup-ready system in one codebase

1.2 Technical Goals

- Sub-500ms P95 latency for queries on a single VPS (Hetzner CX32 or equivalent)
- Adaptive retrieval: system learns which strategy (dense, sparse, hybrid) performs best per query class
- Self-evaluation: every query is scored by RAGAS metrics and logged; scores drive strategy weight updates
- Zero-downtime deploys via rolling Docker Compose restarts
- Full distributed tracing across every pipeline stage

1.3 Functional Requirements

Feature	Description	Priority
Document ingestion	PDF, DOCX, TXT, MD, URL, web scrape	P0
Semantic chunking	Multiple strategies: fixed, sentence, parent-child	P0
Hybrid retrieval	Dense (Weaviate) + Sparse (OpenSearch BM25)	P0
Query routing	Rule-based → ML classifier → self-learning	P0
Cross-encoder reranking	BGE reranker or Cohere rerank	P0
Context compression	LLMLingua or sentence relevance scoring	P1
RAGAS evaluation	Per-query faithfulness, relevancy, recall scores	P0

Semantic caching	Cosine similarity cache in Redis + Weaviate	P1
Agentic workflow	LangGraph supervisor + specialized agents	P1
FastHTML dashboard	Live eval metrics, query logs, strategy distribution	P1
Observability	OpenTelemetry traces, Prometheus metrics, Grafana	P1
CI/CD pipeline	GitHub Actions: test → build → SSH deploy	P0

1.4 Non-Functional Requirements

Latency (P50)	< 200ms retrieval, < 800ms end-to-end
Latency (P95)	
Availability	99.5% uptime on single VPS (no HA in v1)
Throughput	50 concurrent queries on Hetzner CX32
Storage	10GB documents, 5GB vectors, 2GB indices
Security	JWT auth, API key rotation, secrets via env files
Observability	100% trace coverage on all query paths
Deployability	< 5min push-to-production via GitHub Actions

1.5 Scalability & Cost Targets

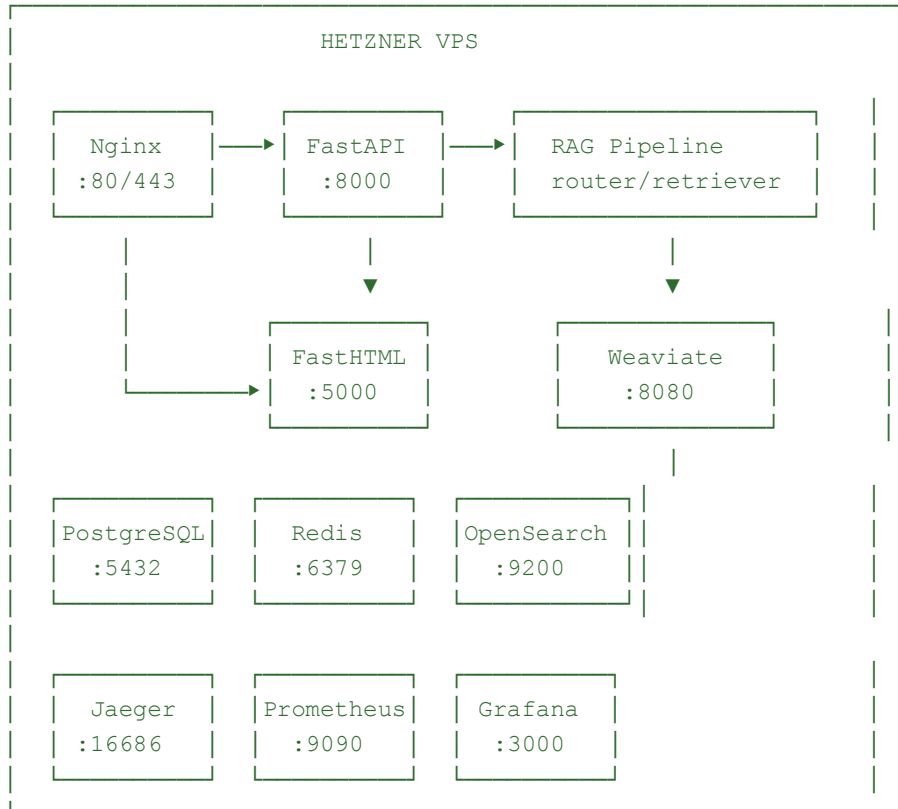
Version 1 targets a single Hetzner CX32 (4 vCPU, 8GB RAM, €13/mo). The architecture is designed so that horizontal scaling (splitting services across VMs or moving to ECS Fargate) requires no code changes — only Docker Compose → ECS migration.

- Weaviate: scales to ~1M vectors on a single node before sharding is needed
- OpenSearch: single node sufficient up to ~50GB index
- PostgreSQL: single node with read replica for scale
- Redis: single node with AOF persistence

Section 2 — High Level Architecture

2.1 System Architecture

The system is composed of 8 core services running in Docker Compose on a single VPS, fronted by Nginx as a reverse proxy and TLS terminator.



2.2 Component Responsibilities

Component	Responsibility	Tech
Nginx	TLS termination, reverse proxy, static files	nginx:alpine
FastAPI	REST API, pipeline orchestration, auth	Python 3.11
FastHTML	Eval dashboard, HTMX-powered UI	Python FastHTML

Weaviate	Vector store, dense + hybrid retrieval	semitechnologies/weaviate
OpenSearch	BM25 sparse retrieval, full-text index	opensearch 2.11
PostgreSQL	Query logs, RAGAS scores, router weights	postgres:16
Redis	Semantic cache, rate limiting, session store	redis:7-alpine
Jaeger	Distributed trace collection and UI	jaegertracing/all-in-one
Prometheus	Metrics scraping and storage	prom/prometheus
Grafana	Metrics dashboards and alerting	grafana/grafana

2.3 Request Lifecycle

A complete query follows this path through the system:

- 1. Client sends POST /api/query with JWT bearer token
- 2. Nginx proxies to FastAPI on port 8000
- 3. FastAPI middleware validates JWT, starts OpenTelemetry trace span
- 4. Semantic cache checked — if hit (cosine similarity > 0.95), return cached response
- 5. Query router classifies query type and selects retrieval strategy
- 6. Dense retrieval from Weaviate and/or sparse BM25 from OpenSearch
- 7. Cross-encoder reranker merges and rescores results
- 8. Context compressor strips irrelevant sentences, trims to token budget
- 9. LLM generates answer (Bedrock Claude / local Ollama)
- 10. RAGAS evaluation runs asynchronously, scores logged to PostgreSQL
- 11. Response returned to client with answer, sources, trace_id, latency, RAGAS scores
- 12. Feedback loop updates router strategy weights based on RAGAS scores

Section 3 — Ingestion Pipeline

3.1 Upload API

The ingestion entry point is a FastAPI endpoint that accepts multipart file uploads or URLs, validates them, and dispatches background processing via Celery workers.

```
POST /api/ingest
Content-Type: multipart/form-data
Body: { file: <binary>, metadata: { source, tags, collection } }

POST /api/ingest/url
Body: { url: 'https://...', depth: 1, metadata: {...} }
```

3.2 Ingestion Worker Pipeline

Each uploaded document flows through a 7-stage pipeline executed by Celery workers:

- Stage 1 — Validation: file type whitelist (PDF, DOCX, TXT, MD), max size 50MB, MIME type check
- Stage 2 — Malware scan: ClamAV via clamd socket (optional, can be skipped in dev)
- Stage 3 — Parsing: format-specific extractors (pdfplumber for PDF, python-docx for DOCX, html2text for URLs)
- Stage 4 — Cleaning: strip headers/footers, remove boilerplate, normalize whitespace, fix encoding
- Stage 5 — Metadata extraction: title, author, date, page count, language detection via langdetect
- Stage 6 — Chunking: strategy selected based on document type (see below)
- Stage 7 — Embedding + Indexing: batch embed chunks, write to Weaviate and OpenSearch in parallel

3.3 Chunking Strategies

Strategy	When to use	Chunk size
Fixed-size	Legal docs, contracts — uniform structure	512 tokens, 50 overlap
Sentence-aware	Articles, blogs — narrative flow	3-5 sentences per chunk
Semantic	Research papers — topic coherence	Variable, split on topic shift
Parent-child	Long docs needing context — retrieve child, pass parent	Child: 256t, Parent: 1024t

Recursive	Mixed-format docs — hierarchical splitting	Splits on <code>\n\n</code> then <code>\n</code> then <code>.</code>
-----------	--	--

Parent-child chunking is the most important strategy for this system. Small child chunks (256 tokens) are indexed for precise retrieval; their parent chunk (1024 tokens) is passed to the LLM for richer context. This significantly improves faithfulness scores.

Section 4 — Embedding Layer

4.1 Embedding Model Selection

Model	Pros	Cons
BGE-M3 (recommended)	Multilingual, dense+sparse+colbert in one, free, self-hosted	1.5GB RAM, slower than API
E5-large-v2	Strong English performance, lighter than BGE-M3	English-only, no sparse
Voyage AI	State-of-art retrieval, 128K context	API cost, external dependency
Bedrock Titan	AWS-native, no infra, good quality	API cost, vendor lock-in
OpenAI text-embedding-3-small	Fast, cheap, good quality	External dependency, cost

Recommendation: Use BGE-M3 self-hosted via a FastAPI embedding microservice. It produces dense, sparse, and ColBERT embeddings simultaneously — eliminating the need for separate BM25 infrastructure if desired. For VPS deployment, it runs on CPU with ~3s per batch of 32 chunks.

4.2 Embedding Versioning

Embedding models change over time. When you upgrade models, existing vectors become incompatible. The solution is embedding versioning in PostgreSQL:

```
documents table: embedding_model_version VARCHAR(50) DEFAULT 'bge-m3-v1'
On model upgrade: background job re-embeds all documents at new version
Weaviate collection names include version: 'Chunks_bge_m3_v1',
'Chunks_bge_m3_v2'
Query router checks version and routes to correct collection
```

4.3 Batch Processing & Caching

- Batch size: 32 chunks per embedding call (optimal for CPU inference with BGE-M3)
- Embedding cache: Redis cache keyed by SHA256(text + model_version), TTL 7 days
- Cache hit rate typically 20-40% during re-ingestion of updated documents
- Async embedding: Celery task, non-blocking for upload API

Section 5 — Storage Architecture

5.1 Weaviate Schema

```
Class: DocumentChunk
Properties:
  chunk_id:      UUID (primary identifier)
  document_id:   UUID (FK to PostgreSQL documents table)
  content:       text (the chunk text)
  parent_id:     UUID (FK to parent chunk, nullable)
  chunk_index:   int (position in document)
  embedding_model: text (e.g. 'bge-m3-v1')
  source:        text (filename or URL)
  tags:          text[] (user-defined tags)
  created_at:    date
Vectorizer: none (external embeddings via Python)
Index: HNSW, ef=128, maxConnections=64
```

5.2 OpenSearch Schema

```
Index: document_chunks
Mappings:
  chunk_id:      keyword
  document_id:   keyword
  content:       text (analyzer: english, BM25 similarity)
  source:        keyword
  tags:          keyword[]
  chunk_index:   integer
  created_at:    date
Settings: number_of_shards: 1, number_of_replicas: 0 (single node)
```

5.3 PostgreSQL Schema

```
Table: documents
  id UUID PRIMARY KEY DEFAULT gen_random_uuid()
  filename TEXT NOT NULL
  source_type TEXT CHECK(source_type IN ('pdf','docx','txt','md','url'))
  embedding_model_version TEXT DEFAULT 'bge-m3-v1'
  status TEXT DEFAULT 'pending'
  chunk_count INT
  created_at TIMESTAMPTZ DEFAULT now()

Table: query_logs
  id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

```

query TEXT NOT NULL
strategy_used TEXT
answer TEXT
latency_ms INT
cache_hit BOOLEAN DEFAULT false
trace_id TEXT
created_at TIMESTAMPTZ DEFAULT now()

```

Table: ragas_scores

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
query_log_id UUID REFERENCES query_logs(id)
faithfulness FLOAT
answer_relevancy FLOAT
context_recall FLOAT
context_precision FLOAT
created_at TIMESTAMPTZ DEFAULT now()

```

Table: router_weights

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
query_class TEXT NOT NULL
strategy TEXT NOT NULL
weight FLOAT DEFAULT 0.33
sample_count INT DEFAULT 0
updated_at TIMESTAMPTZ DEFAULT now()

```

5.4 Redis Schema

semantic_cache:{query_hash}	JSON: {answer, sources, ragas, created_at} — TTL 1hr
embedding_cache:{sha256}	Serialized numpy array — TTL 7 days
rate_limit:{api_key}	Counter — TTL 60s (sliding window)
router_weights:{query_class}	JSON weights — TTL 5min (refreshed from Postgres)
query_lock:{query_hash}	Mutex to prevent cache stampede — TTL 10s

Section 6 — Retrieval Architecture

6.1 Dense Retrieval

Dense retrieval uses Weaviate's HNSW index to perform approximate nearest neighbour search over embedded query vectors.

Query flow:

1. Embed query using BGE-M3 (or cached embedding)
2. Weaviate nearVector search with top_k=20
3. Optional metadata filter: {tags: [...], source: '...'}
4. Returns: [{chunk_id, content, score, document_id}]

6.2 Sparse Retrieval (BM25)

Sparse retrieval uses OpenSearch BM25 which excels at keyword-heavy and exact-match queries.

Query flow:

1. Tokenize query (no embedding needed)
2. OpenSearch multi_match query across content field
3. BM25 scoring: $\text{score}(D, Q) = \sum \text{IDF}(q_i) * f(q_i, D)^{(k_1+1)} / (f(q_i, D) + k_1 * (1 - b + b * |D| / \text{avgdl}))$
4. top_k=20 results returned

6.3 Hybrid Fusion — Reciprocal Rank Fusion

When both retrievers are used, results are merged using Reciprocal Rank Fusion (RRF):

$\text{RRF_score}(d) = \sum 1 / (k + \text{rank}_i(d))$
where $k=60$ (standard), $\text{rank}_i(d)$ = rank of document d in retriever i

Example:

Dense rank of chunk_A: 3 → $1 / (60 + 3) = 0.01587$
Sparse rank of chunk_A: 8 → $1 / (60 + 8) = 0.01471$
RRF score of chunk_A: $0.01587 + 0.01471 = 0.03058$

RRF is preferred over weighted score fusion because it is robust to score scale differences between dense (cosine 0-1) and sparse (BM25 unbounded) retrievers.

Section 7 — Query Router

7.1 Version 1 — Rule-Based Router

The simplest router uses heuristic rules to classify query type and select strategy:

```
def route_v1(query: str) -> str:
    tokens = query.lower().split()
    # Keyword signals → sparse
    if any(t in tokens for t in ['code', 'function', 'error', 'version',
    'syntax']):
        return 'sparse'
    # Conceptual signals → dense
    if any(t in tokens for t in ['explain', 'why', 'how', 'concept', 'mean']):
        return 'dense'
    # Default → hybrid
    return 'hybrid'
```

7.2 Version 2 — ML Classifier Router

After collecting 500+ query logs with RAGAS scores, train a logistic regression classifier:

```
Features:
- query length (tokens)
- avg word length (longer = more technical = sparse)
- question word (what/why/how/when/which)
- contains quoted string (exact match → sparse)
- entity count via spaCy NER
- sentence count

Label: best_strategy (strategy that produced highest RAGAS faithfulness for
similar queries)
Model: sklearn LogisticRegression, C=1.0, max_iter=1000
Retrain: weekly GitHub Actions job
```

7.3 Version 3 — LLM Router

For complex query classification, use a fast LLM with a structured prompt:

```
System: You are a retrieval strategy classifier. Given a query, output JSON:
    {strategy: 'dense' | 'sparse' | 'hybrid', confidence: 0.0-1.0, reasoning: str}
Use dense for conceptual/semantic queries.
Use sparse for exact keyword/code/name queries.
Use hybrid for mixed queries.
```

7.4 Version 4 — Self-Learning Router

The self-learning router uses a multi-armed bandit (Thompson Sampling) to continuously update strategy selection probabilities:

```
For each query class c and strategy s:  
    alpha[c][s] = successes (RAGAS faithfulness > 0.8)  
    beta[c][s]  = failures  (RAGAS faithfulness <= 0.8)  
    sample from Beta(alpha, beta) distribution  
    select strategy with highest sample  
    update alpha/beta after each query based on RAGAS score
```

Section 8 — Reranking

8.1 Cross-Encoder Reranker

After retrieval, a cross-encoder scores (query, chunk) pairs jointly, producing much more accurate relevance scores than bi-encoder retrieval alone.

Model options:

- BAAI/bge-reranker-v2-m3 (recommended, multilingual, free)
- cross-encoder/ms-marco-MiniLM-L-6-v2 (fast, English)
- Cohere rerank-english-v3.0 (API, highest quality, \$0.002/1K chunks)

8.2 Latency vs Accuracy Tradeoffs

Reranker	Latency (top-20)	Quality
No reranker	0ms	Baseline
MiniLM-L-6 (CPU)	~80ms	+12% NDCG
BGE-reranker-v2-m3 (CPU)	~200ms	+18% NDCG
Cohere rerank-v3 (API)	~300ms	+22% NDCG

Production recommendation: use BGE-reranker-v2-m3 self-hosted. Run it as a separate FastAPI microservice within Docker Compose. This isolates the CPU-heavy reranking from the main API process.

Section 9 — Context Compression

9.1 Why Context Compression Matters

Top-k retrieval typically returns 5-10 chunks totalling 2000-5000 tokens. Passing all of this to the LLM increases cost and can hurt quality (lost-in-the-middle problem). Context compression strips irrelevant content before LLM generation.

9.2 Compression Strategies

Strategy	Method	Token reduction
LLMLingua	Small LLM scores token importance, prunes low-importance tokens	50-70%
Sentence relevance filter	Embed sentences, cosine similarity to query, keep top-N	30-50%
Extractive summary	TextRank / BERTSumExt on each chunk	40-60%
Token budget hard cap	Truncate to max_context_tokens=3000, keep highest-ranked chunks	Variable

Recommendation: Sentence relevance filter is the best tradeoff for a VPS deployment. LLMLingua requires an additional model and adds latency. The sentence filter adds ~30ms and reliably reduces context by 40%.

9.3 Algorithm

```
def compress_context(query: str, chunks: list[str], max_tokens: int = 3000):
    query_emb = embed(query)
    sentences = [s for chunk in chunks for s in split_sentences(chunk)]
    sent_embs = embed_batch(sentences)
    scores = cosine_similarity(query_emb, sent_embs)
    ranked = sorted(zip(sentences, scores), key=lambda x: -x[1])
    selected, token_count = [], 0
    for sent, score in ranked:
        tokens = count_tokens(sent)
        if token_count + tokens > max_tokens: break
        selected.append(sent)
        token_count += tokens
    return ' '.join(selected)
```

Section 10 — Generation Layer

10.1 Model Routing Strategy

Model	Use case	Cost
Bedrock Claude Sonnet	Production, high quality, complex queries	~\$0.003/query
Ollama qwen2.5:7b	Dev, offline, free, fast on CPU	\$0
Ollama llama3.1:8b	Good quality local fallback	\$0
GPT-4o-mini	Cheap API fallback	~\$0.001/query

10.2 Fallback Chain

```
Primary:    Bedrock Claude Sonnet
Fallback1:  Ollama qwen2.5:7b (if Bedrock fails or rate-limited)
Fallback2:  Return cached similar answer (if Ollama fails)
Fallback3:  Return error with trace_id for debugging
```

10.3 Core Prompt Template

```
SYSTEM:
You are a precise document assistant. Answer ONLY from the provided context.
If the answer is not in the context, say 'I don't have enough information.'
Always cite your sources using [Source: {filename}] notation.

USER:
Context:
{compressed_context}

Question: {query}

Answer concisely and cite sources.
```

10.4 Retry Policy

```
@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=10))
async def generate(prompt: str, model: str) -> str:
    # exponential backoff: 1s, 2s, 4s
    # on 429 (rate limit): wait for Retry-After header
    # on 500: immediate retry
```



```
# on timeout (>30s): switch to fallback model
```

Section 11 — Agentic Architecture (LangGraph)

11.1 Agent Overview

The agentic layer sits on top of the core RAG pipeline and handles complex multi-step queries that require planning, tool use, and synthesis across multiple retrievals.

11.2 Agent Roles

Agent	Responsibility	Tools
Supervisor	Decomposes query, routes to agents, merges results	route_task, merge_results
Retrieval Agent	Executes RAG pipeline for a sub-query	dense_retrieve, sparse_retrieve, rerank
Search Agent	Web search for current information	web_search (optional)
Evaluation Agent	Scores intermediate answers with RAGAS	ragas_score
Citation Agent	Extracts and formats citations from sources	extract_citations
Summarization Agent	Condenses multiple retrieved answers	summarize

11.3 LangGraph State Schema

```
class RAGState(TypedDict):
    original_query: str
    sub_queries: list[str]
    retrieved_chunks: dict[str, list[Chunk]] # sub_query → chunks
    intermediate_answers: dict[str, str]
    ragas_scores: dict[str, float]
    final_answer: str
    citations: list[Citation]
    trace_id: str
    error: Optional[str]
```

11.4 Graph Transitions

```
START → supervisor → [retrieval_agent, search_agent] (parallel)
retrieval_agent → evaluation_agent → (score < threshold) → retrieval_agent
```

```
                                → (score >= threshold) → citation_agent  
search_agent → summarization_agent → citation_agent  
citation_agent → supervisor (merge) → END
```

Failure handling: if any agent fails 3 times, the supervisor routes to a fallback path that returns the best available answer with a confidence warning.

Section 12 — Evaluation System

12.1 RAGAS Metrics

Metric	Measures	Target
Faithfulness	Is the answer grounded in retrieved context?	> 0.80
Answer Relevancy	Does the answer address the question?	> 0.85
Context Recall	Does retrieved context contain the answer?	> 0.75
Context Precision	Is retrieved context relevant (no noise)?	> 0.70

12.2 Online vs Offline Evaluation

Online evaluation	Runs asynchronously on every production query. RAGAS scores computed in background Celery task after response is returned to user. Scores logged to PostgreSQL. No latency impact.
Offline evaluation	Weekly batch run against a golden dataset of 100 Q&A pairs with known ground-truth answers. Alerts if any metric drops > 5% from baseline. Triggered by GitHub Actions.

12.3 Golden Dataset

The golden dataset is 100 manually curated question-answer pairs covering diverse query types: factual, conceptual, multi-hop, keyword-heavy, and ambiguous. It is stored in PostgreSQL and version-controlled in the repo as a JSON fixture. New pairs are added when production queries reveal gaps.

Section 13 — Feedback Learning Loop

13.1 Logging Strategy

Every query logs the following to PostgreSQL: query text, strategy used, retrieved chunk IDs, answer, latency, RAGAS scores, model used, cache hit, trace ID. This creates a rich dataset for continuous improvement.

13.2 Feedback Collection

Two feedback sources feed the learning loop:

- Automatic: RAGAS scores computed for every query. High faithfulness = positive signal for the chosen strategy.
- Explicit: FastHTML dashboard allows marking answers as correct/incorrect. These override RAGAS signals.

13.3 Strategy Weight Update

```
# Runs nightly via GitHub Actions cron job
def update_router_weights():
    # Query last 24h of logs
    logs = db.query('SELECT query_class, strategy_used, faithfulness
                     FROM query_logs JOIN ragas_scores ...'
                     WHERE created_at > now() - interval '24 hours')

    # Compute per-class, per-strategy mean faithfulness
    for query_class in query_classes:
        for strategy in strategies:
            score = mean([l.faithfulness for l in logs
                          if l.query_class==query_class and
                          l.strategy==strategy])
            db.upsert(router_weights, {query_class, strategy, weight: score})
            redis.delete('router_weights:*') # invalidate cache
```

Section 14 — Semantic Cache

14.1 How Semantic Caching Works

Unlike exact-match caching (which only hits on identical queries), semantic caching uses embedding similarity to find near-duplicate queries and return cached answers.

```
def semantic_cache_lookup(query: str) -> Optional[CachedResponse]:
    query_emb = embed(query)
    # Search Weaviate for similar past queries
    results = weaviate.query('CachedQuery')
        .with_near_vector({'vector': query_emb})
        .with_limit(1)
        .with_additional(['certainty'])
    if results and results[0]['_additional']['certainty'] > 0.95:
        cache_key = results[0]['cache_key']
        return redis.get(f'semantic_cache:{cache_key}')
    return None
```

14.2 Cache Invalidation

TTL-based	All cache entries expire after 1 hour. Simple, safe default.
Document-triggered	When a document is updated/deleted, all cache entries referencing its chunk_ids are invalidated.
Score-triggered	If a cached answer has RAGAS faithfulness < 0.6 on re-evaluation, it is evicted.
Manual	FastHTML dashboard allows clearing cache by collection or globally.

14.3 Cache Stampede Prevention

When a popular cache entry expires, multiple concurrent requests can simultaneously trigger expensive LLM calls. Prevention using Redis mutex:

```
lock = redis.set(f'query_lock:{query_hash}', 1, nx=True, ex=10)
if lock: compute_and_cache() # winner computes
else: wait_for_cache(query_hash) # others wait up to 10s
```

Section 15 — Observability

15.1 OpenTelemetry Tracing

Every query generates a trace spanning all pipeline stages. Each stage is a span with attributes:

Spans per query:

<code>rag.query</code>	- root span, query text, strategy_used
<code>rag.cache_lookup</code>	- cache hit/miss
<code>rag.router</code>	- query_class, strategy selected
<code>rag.dense_retrieve</code>	- top_k, latency_ms, num_results
<code>rag.sparse_retrieve</code>	- top_k, latency_ms, num_results
<code>rag.rerank</code>	- input_count, output_count, latency_ms
<code>rag.compress</code>	- input_tokens, output_tokens, latency_ms
<code>rag.generate</code>	- model, prompt_tokens, completion_tokens, latency_ms
<code>rag.evaluate</code>	- faithfulness, answer_relevancy, context_recall

15.2 Prometheus Metrics

<code>rag_query_total</code>	Counter	- total queries by strategy
<code>rag_query_latency_seconds</code>	Histogram	- P50/P95/P99 per strategy
<code>rag_cache_hit_total</code>	Counter	- cache hits vs misses
<code>rag_ragas_faithfulness</code>	Gauge	- rolling 1hr mean faithfulness
<code>rag_retrieval_latency</code>	Histogram	- retrieval latency by type
<code>rag_reranker_latency</code>	Histogram	- reranker latency
<code>rag_llm_tokens_total</code>	Counter	- tokens consumed by model

15.3 Grafana Dashboards

- Dashboard 1 — Query Overview: queries/min, P95 latency, cache hit rate, error rate
- Dashboard 2 — Retrieval Health: strategy distribution, retrieval latency by type, top-k distribution
- Dashboard 3 — RAGAS Scores: faithfulness trend, answer relevancy trend, alerts on drop > 5%
- Dashboard 4 — System Health: CPU, RAM, disk per container, Weaviate/OpenSearch/Redis health
- Dashboard 5 — LLM Usage: tokens/hr, model distribution, fallback rate, cost estimate

Section 16 — Security

16.1 Authentication & Authorization

Layer	Mechanism	Implementation
API authentication	JWT bearer tokens	python-jose, 24hr expiry
API keys	SHA256-hashed keys in PostgreSQL	X-API-Key header
Dashboard auth	Session cookies via FastHTML	httponly, secure, samesite=strict
Service-to-service	Internal Docker network (no external exposure)	Docker Compose networks
Secrets	Environment variables via .env file	Never committed to git

16.2 Rate Limiting

```
# Redis sliding window rate limiter
# 100 requests/minute per API key
def check_rate_limit(api_key: str) -> bool:
    key = f'rate_limit:{api_key}'
    count = redis.incr(key)
    if count == 1: redis.expire(key, 60)
    return count <= 100
```

16.3 Security Checklist

- Nginx: X-Frame-Options, X-Content-Type-Options, CSP headers
- PostgreSQL: strong password, no public network exposure
- Weaviate: anonymous access disabled in production, API key auth enabled
- OpenSearch: security plugin enabled with basic auth
- Redis: requirepass set, no public exposure
- All services: only on internal Docker network, not published to host except via Nginx
- TLS: Let's Encrypt via Certbot, auto-renewal via cron

Section 17 — CI/CD Pipeline

17.1 GitHub Actions Workflow

Triggers: push to main, PR to main

Jobs:

1. test
 - pytest unit tests (retrieval, router, compression)
 - pytest integration tests (full pipeline with test fixtures)
 - RAGAS golden dataset evaluation (fail if faithfulness < 0.75)
 - coverage report (fail if < 80%)
2. build (on test pass)
 - docker build for api and dashboard images
 - push to GitHub Container Registry (ghcr.io)
3. deploy (on main branch only)
 - SSH to VPS
 - git pull
 - docker compose pull
 - docker compose up -d --build --no-deps api dashboard
 - docker compose exec api python -m pytest tests/smoke/
 - on failure: docker compose rollback (previous image tag)

17.2 Rollback Strategy

```
# Every deploy tags images with git SHA
# Rollback = re-deploy previous SHA
docker compose pull api:${PREVIOUS_SHA}
docker compose up -d --no-deps api
```

Section 18 — Docker Architecture

18.1 Complete docker-compose.yml

```
services:
  nginx:
    image: nginx:alpine
    ports: ['80:80', '443:443']
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - /etc/letsencrypt:/etc/letsencrypt:ro
    depends_on: [api, dashboard]

  api:
    build: { context: ., dockerfile: Dockerfile.api }
    environment:
      - WEAVIATE_URL=http://weaviate:8080
      - OPENSEARCH_URL=http://opensearch:9200
      - REDIS_URL=redis://redis:6379
      - DATABASE_URL=postgresql://rag:secret@postgres:5432/ragdb
      - OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317
    depends_on: [weaviate, opensearch, redis, postgres]

  dashboard:
    build: { context: ./dashboard, dockerfile: Dockerfile }
    environment:
      - API_URL=http://api:8000
      - DATABASE_URL=postgresql://rag:secret@postgres:5432/ragdb

  weaviate:
    image: semitechnologies/weaviate:1.24.0
    volumes: [weaviate-data:/var/lib/weaviate]
    environment:
      QUERY_DEFAULTS_LIMIT: 25
      AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED: 'false'
      AUTHENTICATION_APIKEY_ENABLED: 'true'
      AUTHENTICATION_APIKEY_ALLOWED_KEYS: '${WEAVIATE_API_KEY}'

  opensearch:
    image: opensearchproject/opensearch:2.11.0
    volumes: [opensearch-data:/usr/share/opensearch/data]
    environment:
      discovery.type: single-node
      OPENSEARCH_INITIAL_ADMIN_PASSWORD: '${OPENSEARCH_PASSWORD}'

  postgres:
```

```
    image: postgres:16-alpine
    volumes: [postgres-data:/var/lib/postgresql/data]
    environment:
      POSTGRES_USER: rag
      POSTGRES_PASSWORD: '${POSTGRES_PASSWORD}'
      POSTGRES_DB: ragdb

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
    volumes: [redis-data:/data]

  jaeger:
    image: jaegertracing/all-in-one:latest
    ports: ['16686:16686'] # UI only, internal

  prometheus:
    image: prom/prometheus:latest
    volumes: [./prometheus.yml:/etc/prometheus/prometheus.yml]

  grafana:
    image: grafana/grafana:latest
    volumes: [grafana-data:/var/lib/grafana]
    environment:
      GF_SECURITY_ADMIN_PASSWORD: '${GRAFANA_PASSWORD}'

  volumes:
    weaviate-data: opensearch-data: postgres-data: redis-data: grafana-data:

  networks:
    default:
      name: rag-internal
```

Section 19 — VPS Deployment

19.1 Server Sizing

Component	RAM requirement	Notes
FastAPI	512MB	Async, lightweight
FastHTML dashboard	256MB	HTMX, minimal
Weaviate	1.5GB	HNSW index in memory
OpenSearch	1.5GB	JVM heap = 512MB min
PostgreSQL	256MB	Shared buffers 128MB
Redis	256MB	With AOF persistence
BGE-M3 embedder	1.5GB	CPU inference
Prometheus+Grafana	512MB	Metrics storage
Jaeger	256MB	Trace storage in-memory
OS + Docker overhead	1GB	
Total	~7.5GB	Hetzner CX32: 8GB RAM

19.2 Recommended VPS

Provider	Hetzner Cloud (cheapest EU)
Instance	CX32 (4 vCPU, 8GB RAM, 80GB SSD)
Cost	~€13/month
OS	Ubuntu 24.04 LTS
Storage	80GB SSD (sufficient for 10GB docs + indices)
Backup	Hetzner snapshot weekly (€0.01/GB/month)
Upgrade path	CX42 (8 vCPU, 16GB RAM) at €26/month when needed

19.3 First Deploy Checklist

- SSH into VPS, install Docker + Docker Compose
- Clone repo, create .env from .env.example
- Run: certbot --nginx -d yourdomain.com (TLS)
- Run: docker compose up -d

- Run: `docker compose exec api alembic upgrade head` (DB migrations)
- Run: `docker compose exec api python scripts/init_weaviate.py` (create schema)
- Run: `docker compose exec api python scripts/init_opensearch.py` (create index)
- Add GitHub secrets: `VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`
- Push to main — CI/CD takes over for all future deploys

Section 20 — Project Folder Structure

```
adaptive-rag-engine/
├── api/                                # FastAPI backend
│   ├── main.py                        # App entry point, middleware
│   ├── routers/
│   │   ├── query.py                  # POST /query
│   │   ├── ingest.py                 # POST /ingest
│   │   └── health.py                 # GET /health
│   ├── pipeline/
│   │   ├── router.py                 # Query router (v1-v4)
│   │   ├── retrieval/
│   │   │   ├── dense.py              # Weaviate retrieval
│   │   │   ├── sparse.py             # OpenSearch BM25
│   │   │   └── fusion.py              # RRF fusion
│   │   ├── reranker.py               # Cross-encoder reranking
│   │   ├── compressor.py             # Context compression
│   │   └── generator.py               # LLM generation + fallback
│   ├── ingestion/
│   │   ├── parser.py                 # PDF/DOCX/URL parsing
│   │   ├── chunker.py                # All chunking strategies
│   │   ├── embedder.py               # BGE-M3 embedding service
│   │   └── indexer.py                # Weaviate + OpenSearch write
│   ├── eval/
│   │   ├── ragas_eval.py             # RAGAS evaluation
│   │   ├── feedback.py               # Strategy weight updates
│   │   └── golden_dataset.py          # Golden Q&A evaluation
│   ├── cache/
│   │   └── semantic_cache.py          # Semantic caching logic
│   ├── agents/
│   │   ├── graph.py                  # LangGraph workflow
│   │   └── agents.py                  # Individual agent definitions
│   ├── models/
│   │   └── schemas.py                 # Pydantic request/response models
│   ├── db/
│   │   ├── postgres.py               # SQLAlchemy models + session
│   │   └── migrations/               # Alembic migration files
│   └── core/
│       ├── config.py                 # Settings from env vars
│       ├── auth.py                   # JWT + API key validation
│       └── telemetry.py               # OpenTelemetry setup
├── dashboard/                         # FastHTML dashboard
│   ├── main.py                       # FastHTML app
│   └── components/                   # HTMX partial views
├── embedder/                          # BGE-M3 microservice
│   └── main.py                       # FastAPI embedding endpoint
└── tests/
```

```
|   └─ unit/                # Pure unit tests
|   └─ integration/         # Pipeline integration tests
|   └─ smoke/               # Post-deploy smoke tests
|   └─ fixtures/           # Golden dataset, test docs
└─ scripts/
|   └─ init_weaviate.py     # Create Weaviate schema
|   └─ init_opensearch.py  # Create OpenSearch index
|   └─ update_router_weights.py # Nightly weight update
└─ infra/
|   └─ nginx.conf
|   └─ prometheus.yml
|   └─ grafana/            # Dashboard JSON exports
└─ .github/workflows/
|   └─ test.yml            # PR tests
|   └─ deploy.yml         # Main branch deploy
└─ docker-compose.yml
└─ docker-compose.dev.yml  # Dev overrides (hot reload)
└─ Dockerfile.api
└─ .env.example
└─ README.md              # Live URL, architecture, setup
```

Section 21 — API Design

21.1 Core Endpoints

Method + Path	Description	Auth
POST /api/query	Execute RAG query	JWT
POST /api/ingest	Upload document	JWT
POST /api/ingest/url	Ingest URL	JWT
GET /api/documents	List ingested documents	JWT
DELETE /api/documents/{id}	Delete document + chunks	JWT
GET /api/health	System health check	None
GET /api/metrics	Prometheus metrics endpoint	Internal
GET /api/eval/scores	RAGAS score history	JWT
GET /api/cache/stats	Cache hit rate, size	JWT
DELETE /api/cache	Clear semantic cache	JWT

21.2 Query Request/Response Schema

```
# Request
class QueryRequest(BaseModel):
    query: str = Field(..., min_length=3, max_length=2000)
    top_k: int = Field(default=5, ge=1, le=20)
    strategy: Literal['auto', 'dense', 'sparse', 'hybrid'] = 'auto'
    collection: Optional[str] = None # filter by doc collection
    use_cache: bool = True
    use_compression: bool = True
    use_reranker: bool = True

# Response
class QueryResponse(BaseModel):
    answer: str
    sources: list[SourceChunk]
    strategy_used: str
    cache_hit: bool
    trace_id: str
    latency_ms: int
    ragas: Optional[RAGASScores] # null if async eval pending

class SourceChunk(BaseModel):
    chunk_id: str
```



```
content: str
document_id: str
source: str
score: float
```

```
class RAGASScores(BaseModel):
    faithfulness: float
    answer_relevancy: float
    context_recall: float
    context_precision: float
```

Section 22 — Load Testing

22.1 k6 Load Test

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 10 }, // ramp up
    { duration: '5m', target: 50 }, // sustained load
    { duration: '2m', target: 100 }, // stress
    { duration: '1m', target: 0 },   // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<2000'], // P95 < 2s
    http_req_failed: ['rate<0.01'],     // < 1% errors
  },
};
```

22.2 Expected Metrics on Hetzner CX32

Metric	Expected	Alert threshold
P50 latency	< 400ms	> 800ms
P95 latency	< 1200ms	> 2000ms
P99 latency	< 2500ms	> 5000ms
Throughput	20-30 RPS sustained	< 10 RPS
Error rate	< 0.5%	> 1%
Cache hit rate	> 25% after warmup	< 10%

Section 23 — Scaling to 100K Users

23.1 Evolution Path

Stage	Users	Architecture
v1 — Single VPS	~100 DAU	Everything on one Hetzner CX32
v2 — Separate services	~1,000 DAU	Embedder on separate VM, Postgres with read replica
v3 — Cloud move	~10,000 DAU	ECS Fargate + RDS + ElastiCache + managed Weaviate
v4 — Full scale	~100,000 DAU	Multi-region, Weaviate cluster, sharded OpenSearch, CDN

23.2 v1 → v2 Bottlenecks

- BGE-M3 embedding is CPU-bound — move to separate VM with GPU or more vCPUs
- Weaviate HNSW index grows with vector count — monitor RAM, upgrade VPS before OOM
- Redis single node — add persistence + backup, consider Redis Sentinel

23.3 v2 → v3 Bottlenecks

- Docker Compose can't scale individual services — migrate to ECS Fargate or Kubernetes
- Single Postgres becomes read bottleneck — add read replica for query logs
- Nginx single point of failure — move to AWS ALB
- Weaviate needs cluster mode for > 5M vectors

23.4 v3 → v4 Bottlenecks

- LLM generation latency at scale — implement request coalescing, batch generation
- RAGAS evaluation adds cost at scale — sample 10% of queries for eval instead of 100%
- OpenSearch needs sharding across multiple nodes for > 100GB index
- Multi-region requires vector replication strategy — Weaviate doesn't support this natively yet

Section 24 — Senior Engineer Review

24.1 Architecture Weaknesses

Risk	Severity	Mitigation
Single VPS = single point of failure	High	Daily Hetzner snapshots, documented recovery runbook
RAGAS eval adds LLM cost per query	Medium	Async eval, sample 20% of queries in production
BGE-M3 on CPU is slow (3s/batch)	Medium	Embedding cache, batch at ingestion not query time
Weaviate + OpenSearch both in RAM	High	CX32 has 8GB — monitor closely, upgrade to CX42 early
No auth on Grafana/Jaeger/Prometheus	High	Put behind Nginx basic auth, not public
OpenTelemetry Jaeger in-memory storage	Medium	Data lost on restart — switch to Jaeger with Cassandra for persistence
No database migrations CI check	Medium	Add alembic check to CI pipeline
LangGraph agentic loop can infinite-loop	High	Max iterations limit (5), circuit breaker per agent

24.2 What Would Change for Production at OpenAI/Stripe Scale

- Replace Docker Compose with Kubernetes (EKS/GKE) — horizontal pod autoscaling per service
- Replace Weaviate self-hosted with Weaviate Cloud or Pinecone Enterprise — managed, SLA-backed
- Replace local BGE-M3 with batched embedding API (Voyage/OpenAI) with circuit breaker
- Replace single Postgres with CockroachDB or Aurora for multi-region writes
- Replace Redis single node with Redis Cluster (6 nodes minimum)
- Add API Gateway (Kong/AWS API GW) for auth, rate limiting, request logging at edge
- Full feature flags (LaunchDarkly) for router strategy rollout — not hardcoded weights
- RAGAS evaluation sampled at 5% of traffic, sent to dedicated eval service
- All secrets in AWS Secrets Manager / HashiCorp Vault — never in .env files

24.3 Final Recommendations

Build in this order:

- Week 1-2: Ingestion pipeline + Weaviate + OpenSearch indexing. Get documents in.
- Week 3: Dense + sparse retrieval + RRF fusion + cross-encoder reranker. Get retrieval right.
- Week 4: FastAPI endpoints + JWT auth + query router v1 (rule-based). Working API.
- Week 5: RAGAS eval loop + PostgreSQL logging + semantic cache. Self-evaluating system.
- Week 6: OpenTelemetry + Prometheus + Grafana. Full observability.
- Week 7: FastHTML dashboard + LangGraph agentic workflow. Polish and agents.
- Week 8: Docker Compose + Nginx + GitHub Actions + VPS deploy. Ship it.

The single most impressive thing you can demo: open Grafana, run 20 queries, show the RAGAS faithfulness trend updating in real-time, then show Jaeger with the full trace breakdown per query. No interviewer at any AI company has seen a student project with that level of production maturity.

END OF DOCUMENT

Adaptive RAG Engine — Production Engineering Design Document v1.0